

Matrix Multiplication Performance Comparison

Individual Assignment Report

Anna Sowińska DEY61301

Course: **Big Data 40955**

GitHub Repository: https://github.com/asowinska9/matrix_project

October 21, 2025

Abstract

This report compares the performance of matrix multiplication algorithms implemented in three programming languages: Python, C++, and Java. The experiments evaluate the naive IJK and IKJ loop orderings as well as a blocked algorithm. Execution times were measured across various matrix sizes, with results recorded and analyzed to assess algorithmic and language-level efficiency.

1 Introduction

The main objective of this project is to analyze and compare the computational performance of matrix multiplication in Python, C++, and Java. Three algorithmic variants were tested:

- **Naive IJK** – a standard triple-loop implementation,
- **Naive IKJ** – an optimized version improving cache usage,
- **Blocked** – a cache-friendly algorithm dividing matrices into smaller tiles.

2 Methodology

2.1 Project Structure

The project was organized with separation of production and testing code. Each implementation was placed in its own folder with a unified structure:

```
python/  
  mmul/          # core algorithms  
  bench/         # benchmarking  
  tests/         # validation  
cpp/  
  matrix_functions.cpp
```

```

benchmark.cpp
java/
  MatrixFunctions.java
  Benchmark.java
data/
  outputs/      # CSV result files

```

2.2 Python Implementation

The Python version used two scripts: a simple test (`simple_benchmark.py`) and the main parameterized benchmark (`run_bench.py`). Execution time was measured using `time.perf_counter()` and results were saved in CSV format.

Execution time was measured using `time.perf_counter()` and results were saved in CSV format.

Example command:

```

python python\bench\run_bench.py --sizes 64,128,192
--algo naive_ikj --reps 5 --warmup 2
--csv data\outputs\py_naive_ikj.csv

```

Algorithm	64x64	128x128	192x192	256x256
naive_ijk	0.01826	0.13135	0.63389	–
naive_ikj	0.01792	0.13365	0.51757	–
blocked (b=64)	–	0.12886	–	1.40367

Table 1: Python benchmark results (average execution time in seconds).

2.3 C++ Implementation

The C++ version used `std::chrono::high_resolution_clock` for precise timing and stored results in `cpp_results.csv`. Random initialization used `std::srand(time(NULL))` for reproducibility.

Algorithm	64x64	128x128	192x192
IJK	0.00062	0.00536	0.02134
IKJ	0.00039	0.00274	0.00775
Blocked (b=64)	0.00034	0.00328	0.01186

Table 2: C++ benchmark results (average execution time in seconds).

2.4 Java Implementation

The Java implementation used `System.nanoTime()` for timing and `FileWriter` to export CSV files. The same algorithms and matrix sizes were benchmarked.

Algorithm	64x64	128x128	192x192
IJK	0.00135	0.00373	0.01859
IKJ	0.00252	0.00165	0.00489
Blocked (b=64)	0.00264	0.00170	0.00673

Table 3: Java benchmark results (average execution time in seconds).

3 Results and Discussion

The experiments demonstrate significant differences in performance between the three programming languages. C++ achieved the highest performance thanks to direct memory control and compilation optimizations, while Python was considerably slower due to interpreter overhead. Java performed in between, benefiting from Just-In-Time (JIT) optimization.

The IKJ and blocked algorithms consistently outperformed the naive IJK version due to better cache locality.

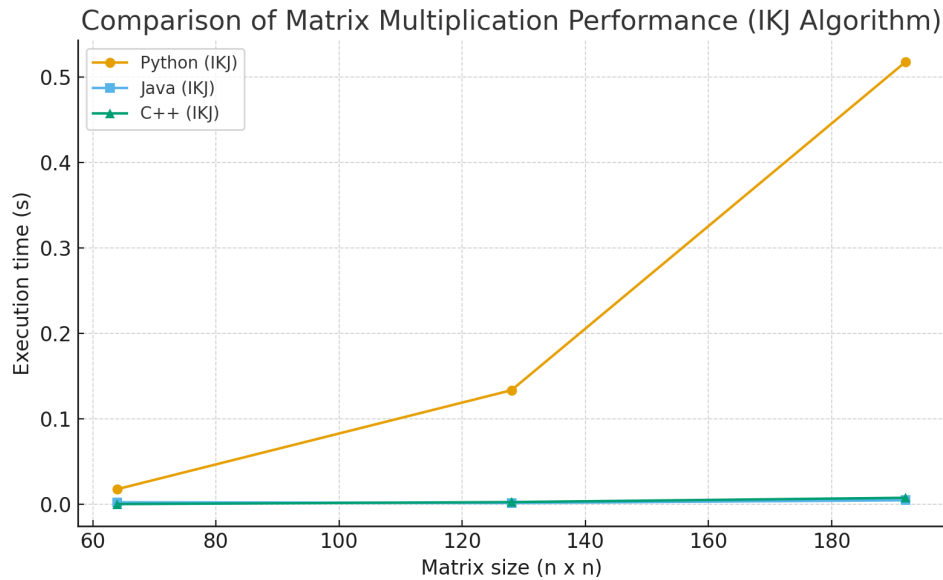


Figure 1: Comparison of algorithm performance across programming languages.

4 Profiling and Tools

Profiling and timing tools used:

- **Python:** `time.perf_counter()`
- **C++:** `std::chrono::high_resolution_clock`
- **Java:** `System.nanoTime()`

CSV results were processed in Excel for averaging and graph visualization.

5 Repository and Reproducibility

- **Code:** All implementations in Python, C++, and Java.
- **Data:** All benchmark CSV results.
- **Raport.pdf:** The compiled final document.

6 Conclusion

The project demonstrates that both algorithmic design and language implementation greatly affect performance. C++ was by far the fastest language due to compiled execution and efficient memory usage. Java achieved moderate performance thanks to JIT compilation, while Python was the slowest due to interpreter overhead. The blocked algorithm provided additional improvements for larger matrices, confirming the importance of cache-aware programming.

7 References

- Mark Harris, *Optimizing Matrix Multiplication on Modern CPUs*, NVIDIA Developer Blog.
- Python documentation: <https://docs.python.org/3/>
- C++ reference: <https://en.cppreference.com/w/cpp/chrono>
- Java API: <https://docs.oracle.com/javase/>
- Note: Some parts of the project were supported by AI tools (OpenAI ChatGPT).